

BCC1313
ALGORITHMS AND COMPLEXITY

Introduction to Algorithms and Complexity

INTRODUCTION

- **Data structures and algorithms** are fundamental concepts in computer science.
- They help **organize and process data efficiently** and **solving problems effectively**.
- Essential for competitive programming, software development, and optimizing code performance.

WHAT IS DATA?

- **Data** refers to all types of information used in programming, from basic numbers to complex strings.
- For example:

```
int num = 20;
double rate = 0.5;
string greeting = "Hello World!";
```

WHAT ARE DATA STRUCTURES?

- **Data structures** define how data is stored, organized, and accessed in memory. Proper organization of data is crucial for the efficiency and functionality of any software.

```
x = "Hello!"
y = "How are you "
z = "today?"
print(x + y + z)
```

Here, three independent strings are stored in variables and combined to create a complete message

THE IMPORTANCE OF DATA ORGANIZATION

- The way data is structured directly **impacts how efficiently a program runs.**
- Efficient organization of data leads to faster performance, especially with large datasets. Poor organization can slow down the system or even cause a program to fail.

REAL-WORLD IMPORTANCE

- Example: If you're building an app that handles large volumes of data or users, choosing the right data structure can determine:
  - whether your application runs smoothly or struggles to handle the load.
  - performance differences between data structures can be dramatic; that is the programs can run orders of magnitude faster with the right choice.

UNDERSTANDING DATA STRUCTURE

- Knowing the right data structure for a given task helps:
  - Write faster - more efficient code.
  - Ensure the software performs well even under heavy loads.
  - Boosts your ability to build scalable and reliable systems.

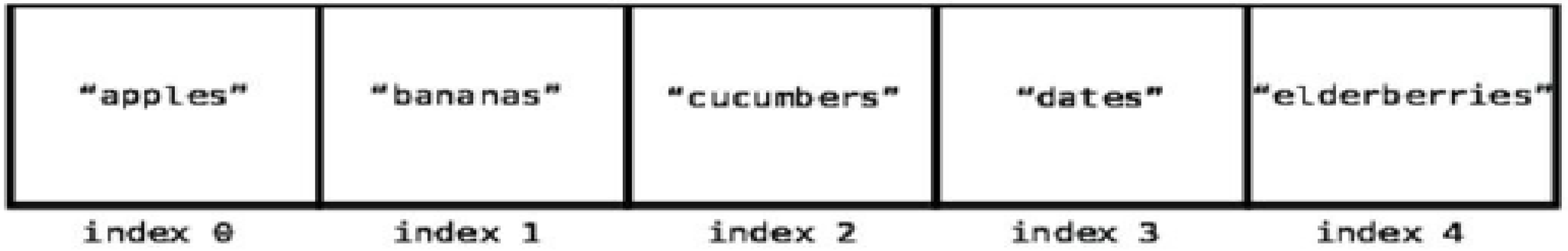
BASIC DATA STRUCTURES

- The two foundational data structures: **arrays** and **sets**.
- Both store collections of elements, but they differ in how they manage and access data.

THE ARRAY

- An array is one of the simplest and most fundamental data structures.
- It is a collection of **elements (data),** often of the **same type**, organized in a **list**. Arrays allow easy access to data using an **index**, where the first element starts at **index 0**.

```
array groceries = [“apples”, “bananas”, “cucumbers”, “dates”, “elderberries”]
```



THE ARRAY

- To understand the performance of array, we need to analyze the common ways that our code might interact with that data structure.
- Most data structures are used in four basic ways (operations):
  - **Read**
  - **Search**
  - **Insert**
  - **Delete**

THE ARRAY

- **Read** - refers to looking something up from a **particular index** within the data structure. For example, looking up which grocery item is located at index 2 would be reading from the array.

```
//answer?
```

- **Search** - refers to looking for a **particular value** within a data structure. For example, looking to see if “dates” is in grocery list, and which index it’s located at would be searching the array.

```
//answer?
```

THE ARRAY

- **Insert** - refers to adding another value to data structure. For example, add “figs” to the list, that would be inserting a new value into the array.

```
//answer?
```

- **Delete** - refers to removing a value from our data structure. For example, removed “bananas” from grocery list, that would be deleting from the array.

```
//answer?
```

THE PERFORMANCE

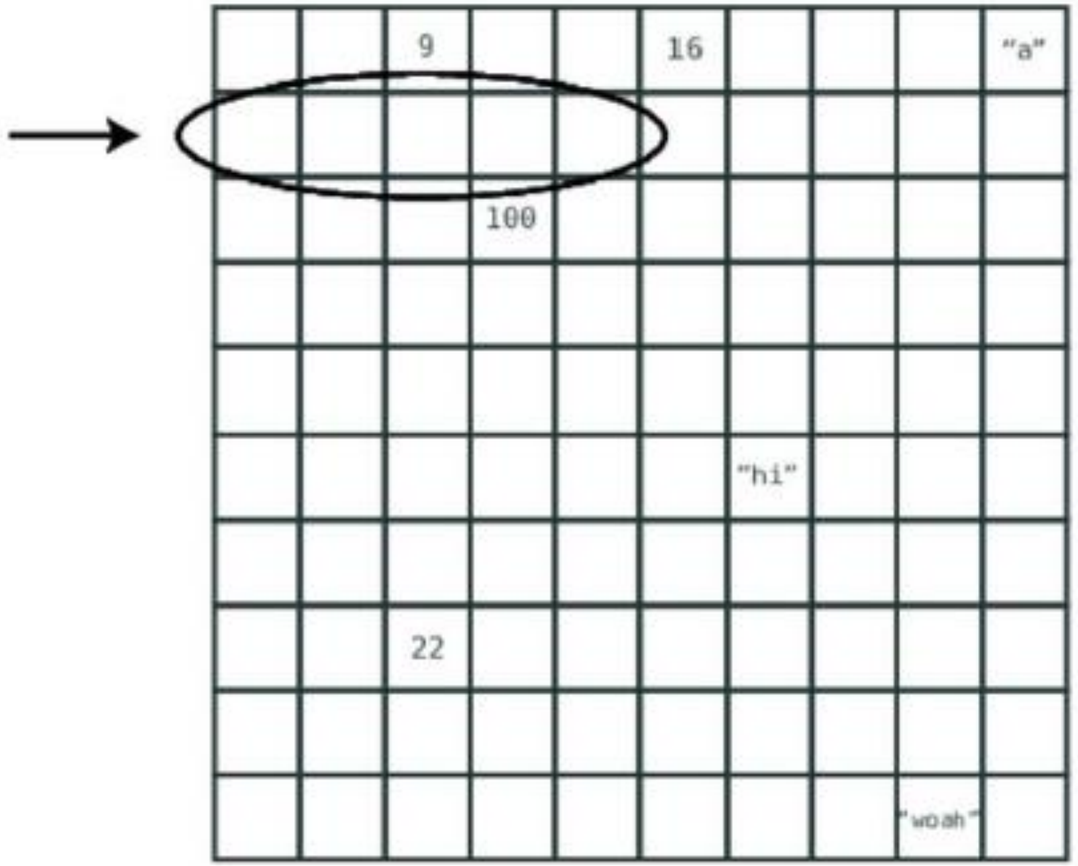
- When we measure how fast an operation takes, we do not refer to how fast the operation takes in terms of pure time, but instead in **how many steps it takes.** For example: if Operation A takes five steps, and Operation B takes 500 steps, we can assume that Operation A will always be faster than Operation B on all pieces of hardware.
- Measuring the speed of an operation is also known as measuring its **time complexity.**

THE PERFORMANCE

- **Read**: Fast, takes one step as the computer can jump to any index directly.
- **Search**: Linear search checks each element one by one, which can be slow if the array is large.
- **Insert**: Inserting at the end is fast, but inserting in the middle or beginning requires shifting elements.
- **Delete**: Similar to insertion, deleting requires shifting elements to avoid gaps in the array.

THE PERFORMANCE

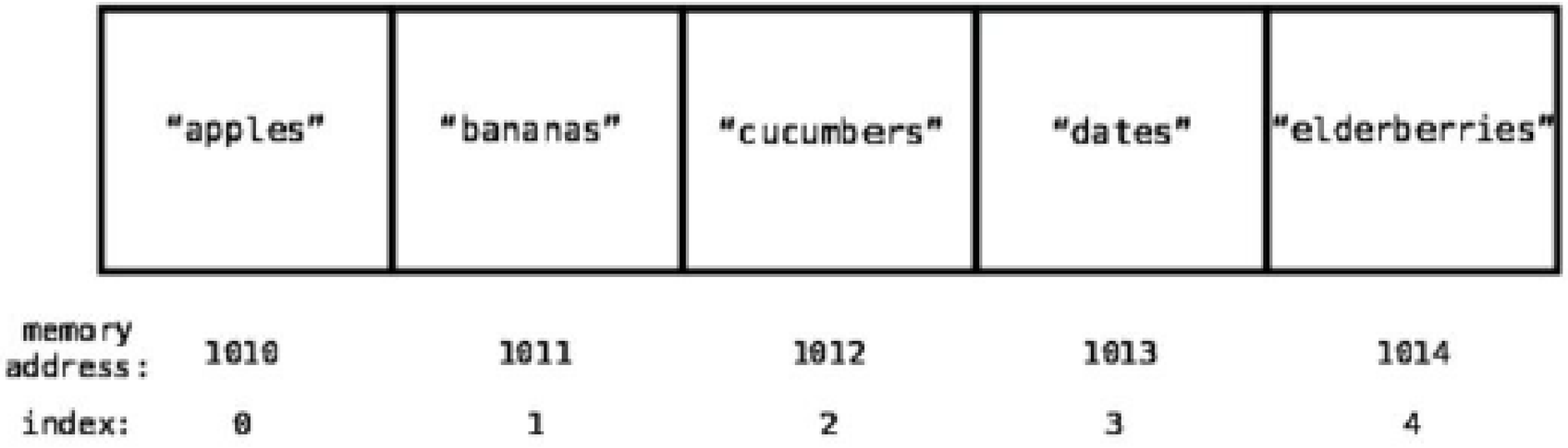
- **Reading Operation**
  - Reading from an array takes just one step. This is because the computer has the ability to jump to any particular index and peer inside.
  - How?



A computer's memory can be viewed as a giant collection of cells. If we are creating an array meant to hold five elements, the computer would find any group of five empty cells in a row and designate it to serve as array

Every cell in a computer's memory has a specific address.

1000	1001	1002	1003	1004	1005	1006	1007	1008	1009
1010	1011	1012	1013	1014	1015	1016	1017	1018	1019
1020	1021	1022	1023	1024	1025	1026	1027	1028	1029
1030	1031	1032	1033	1034	1035	1036	1037	1038	1039
1040	1041	1042	1043	1044	1045	1046	1047	1048	1049
1050	1051	1052	1053	1054	1055	1056	1057	1058	1059
1060	1061	1062	1063	1064	1065	1066	1067	1068	1069
1070	1071	1072	1073	1074	1075	1076	1077	1078	1079
1080	1081	1082	1083	1084	1085	1086	1087	1088	1089
1090	1091	1092	1093	1094	1095	1096	1097	1098	1099



If we tell the computer to read the value at index 3, the computer goes through the following thought process:

1. Our array begins with index 0 at memory address 1010.
2. Index 3 will be exactly three slots past index 0.
3. So to find index 3, we'd go to memory address 1013, since 1010 + 3 is 1013.

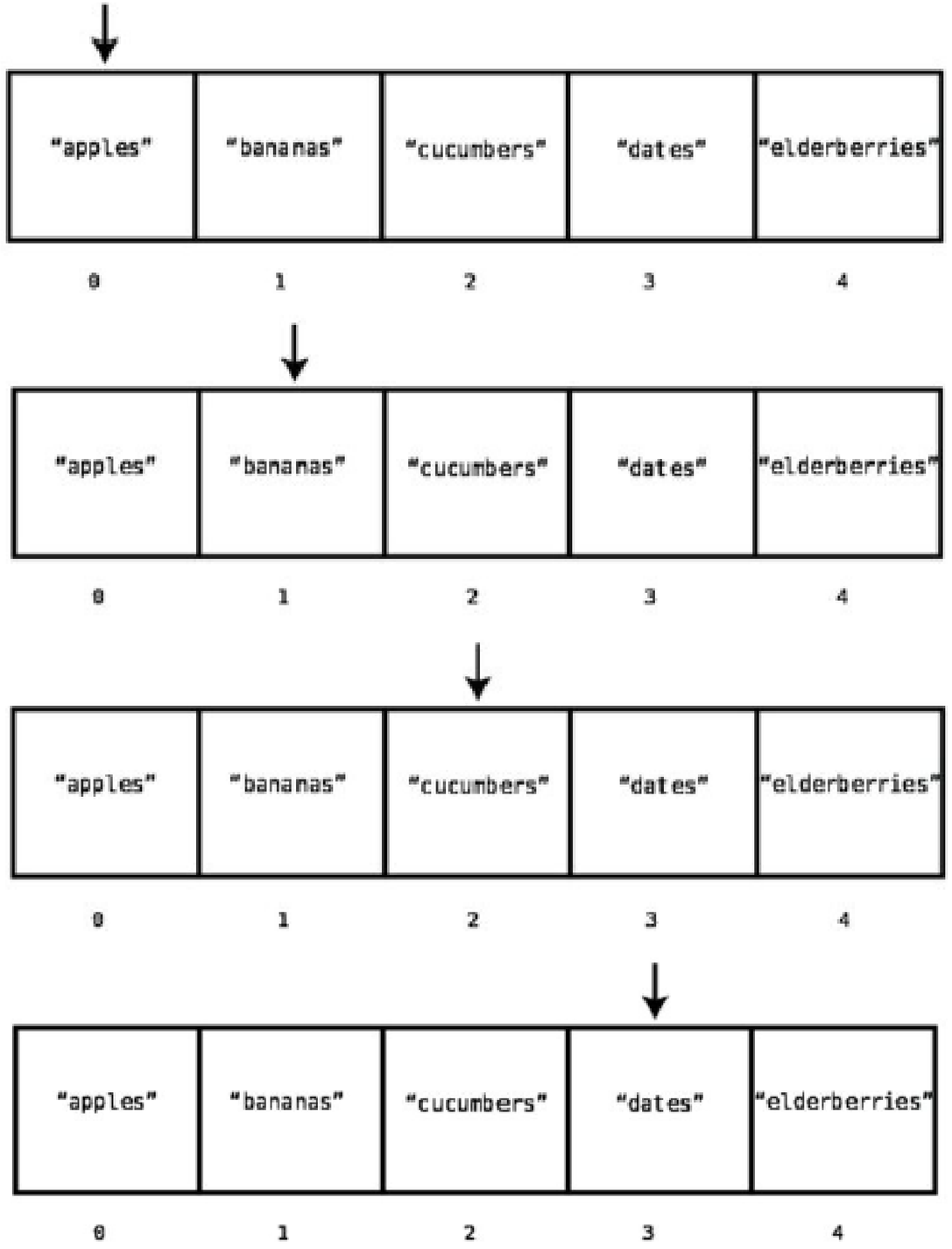
Once the computer jumps to the memory address 1013, it returns the value, which is "dates".

THE PERFORMANCE

- **Searching Operation**
  - Searching an array is looking to see whether a particular value exists within an array and if so, which index it’s located at. How?
  - To search for a value within an array, the computer starts at index 0, checks the value, and if it doesn’t find what it’s looking for, moves on to the next index. It does this until it finds the value it’s seeking.

The following diagrams demonstrate this process when the computer searches for “dates” inside the grocery list array.

In this example, since we had to check four different cells until we found the value, we’d say that this particular operation took a total of four steps.

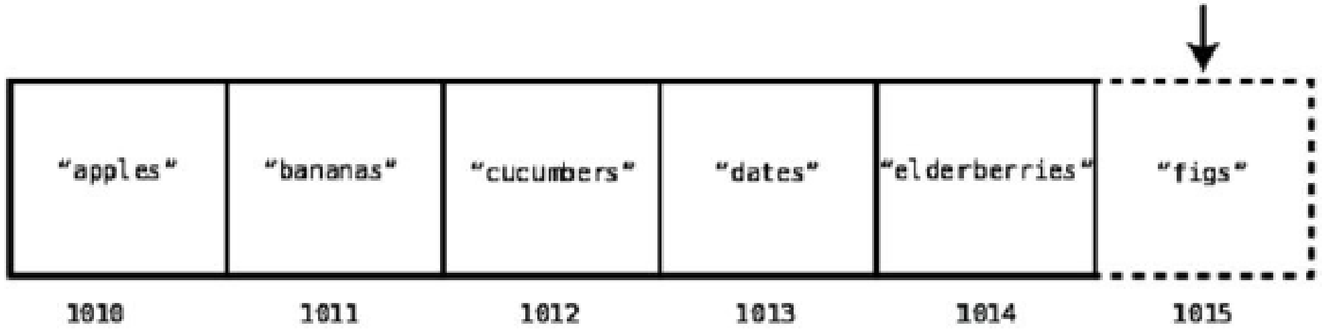


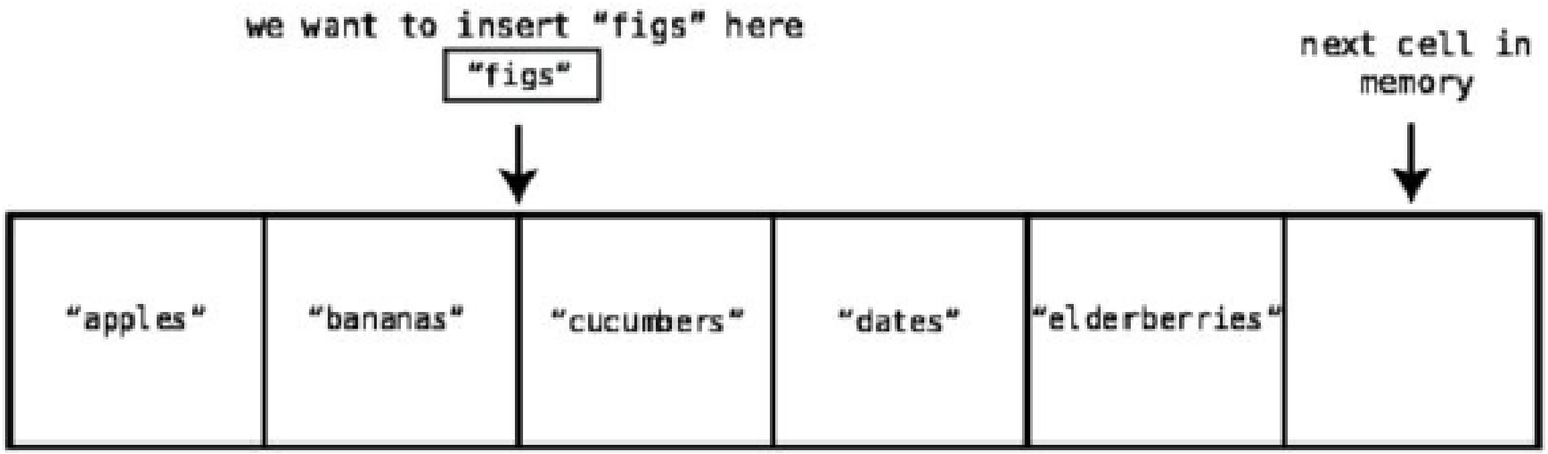
THE PERFORMANCE

- **Insertion Operation**
  - The efficiency of inserting a new piece of data inside an array depends on where inside the array to insert it.

Inserting “figs” to the very end of the list.

The computer knows how many elements the array currently contains, so it can calculate which memory address it needs to add the new element to.

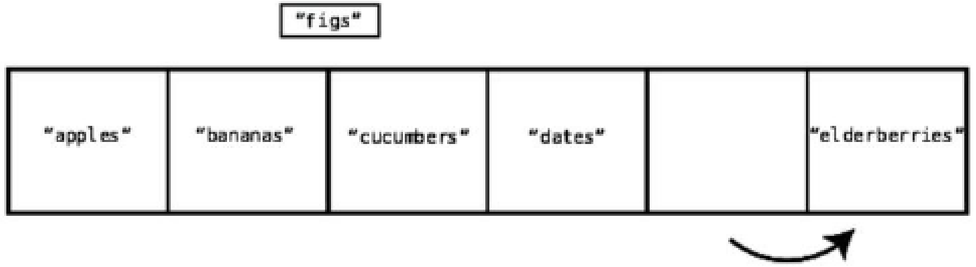




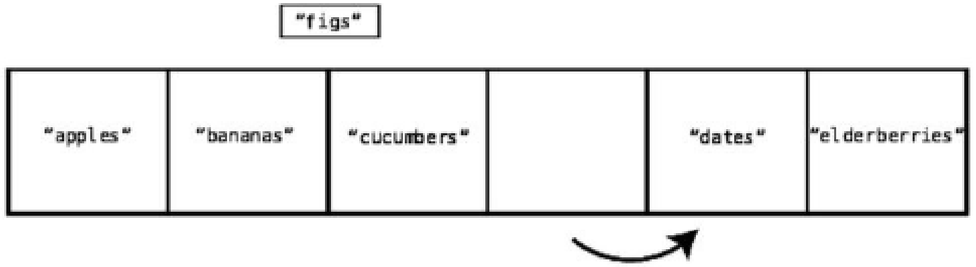
Inserting a new piece of data “figs” **to index 2** within the array.

To do this, we need to move “cucumbers”, “dates”, and “elderberries” to the right to make room for the “figs”. But even this takes multiple steps, since we need to first move “elderberries” one cell to the right to make room to move “dates”. We then need to move “dates” to make room for the “cucumbers”. Let’s walk through this process.

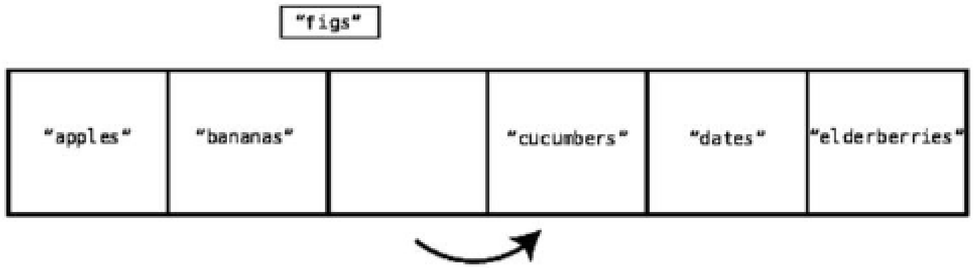
Step #1: We move "elderberries" to the right:



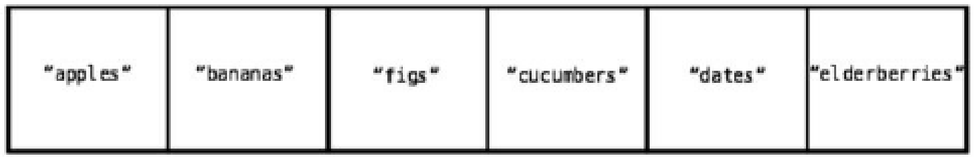
Step #2: We move "dates" to the right:



Step #3: We move "cucumbers" to the right:



Step #4: Finally, we can insert "figs" into index 2:



We can see that there were four steps.

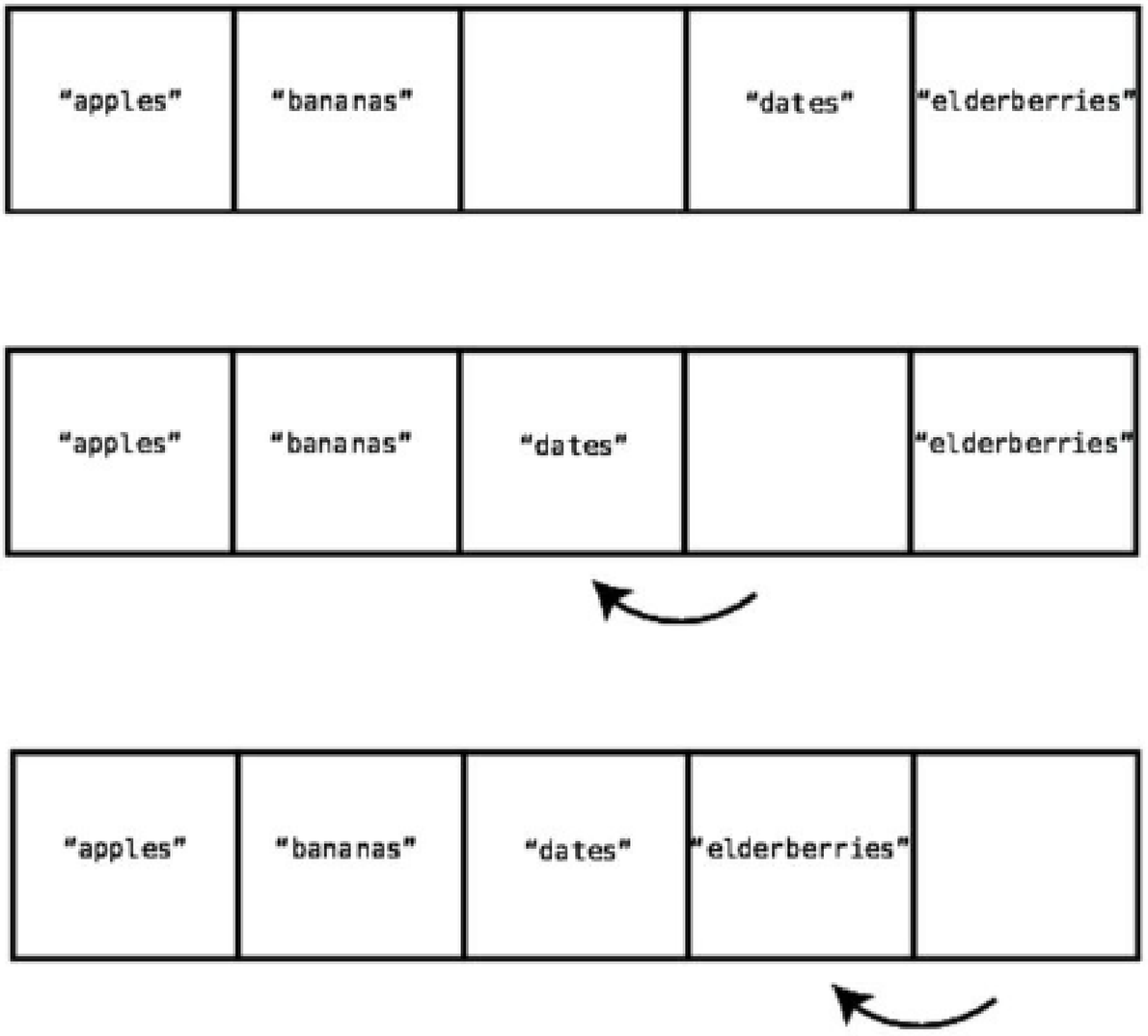
Three of the steps were shifting data to the right, while one step was the actual insertion of the new value.

The worst-case scenario for insertion into an array - that is, the scenario in which insertion takes the most steps is where we insert data at the **beginning** of the array.

This is because when inserting into the beginning of the array, we have to **move all** the other values one cell to the right.

THE PERFORMANCE

- **Deletion Operation**
  - Deletion from an array is the process of eliminating the value at a particular index.



The actual deletion of “cucumbers” technically took just one step.

Problem to solve is an empty cell sitting smack in the middle of the array. An array is not allowed to have gaps, so to resolve this issue, need to shift “dates” and “elderberries” to the left.

The worst-case scenario of deleting an element is deleting the very first element of the array. This is because index 0 would be empty, and we’d have to shift all the remaining elements to the left to fill the gap.

SET

- A **set** is a data structure that does **not allow duplicate values** to be contained within it.
- Sets are useful when you need to ensure that you don’t have duplicate data. For instance, if you’re creating an online phone book, you don’t want the same phone number appearing twice.

SET

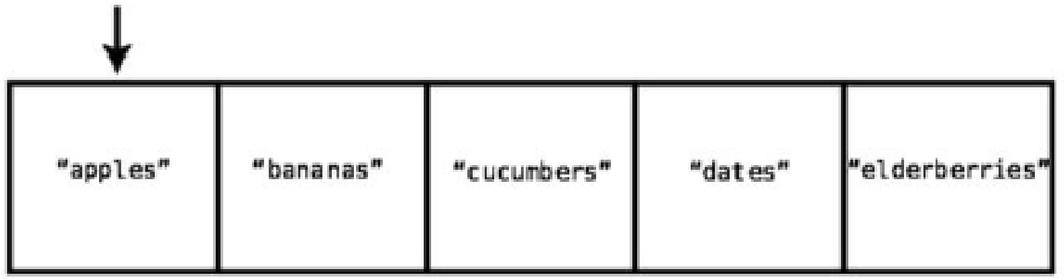
- **Reading** from a set is exactly the same as reading from an array. It takes just one step to look up what is contained within a particular index.
- **Searching** a set also turns out to be no different than searching an array. It takes up to N steps to search to see if a value exists within a set.
- **Deletion** is also identical between a set and an array. It takes up to N steps to delete a value and move data to the left to close the gap.

SET

- **Insertion**, however, is where arrays and sets diverge. With a set, however, the computer first needs to determine that this value doesn’t already exist in this set because that’s what sets do.
- So every insert first requires a search.

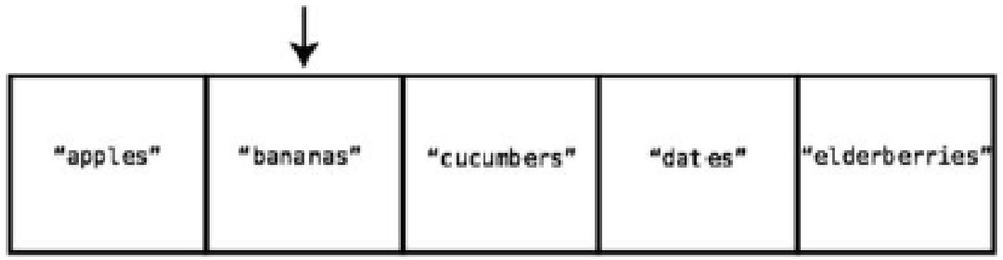
Inserting “figs” into the set.

Step #1: Search index 0 for “figs”:

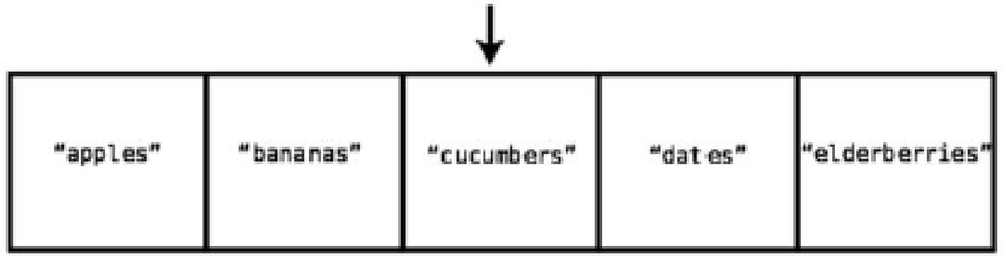


It’s not there, but it might be somewhere else in the set. We need to make sure that “figs” does not exist anywhere before we can insert it.

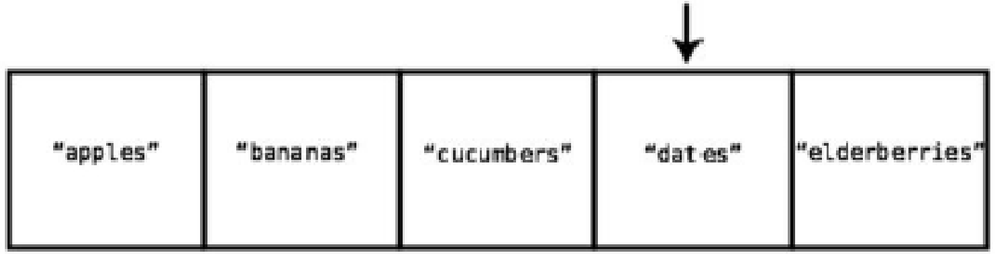
Step #2: Search index 1:



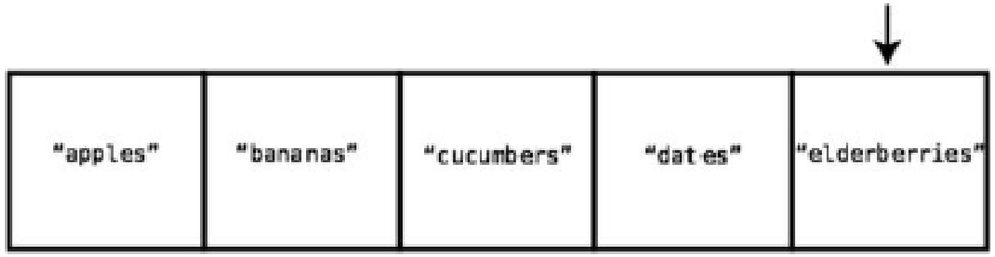
Step #3: Search index 2:



Step #4: Search index 3:

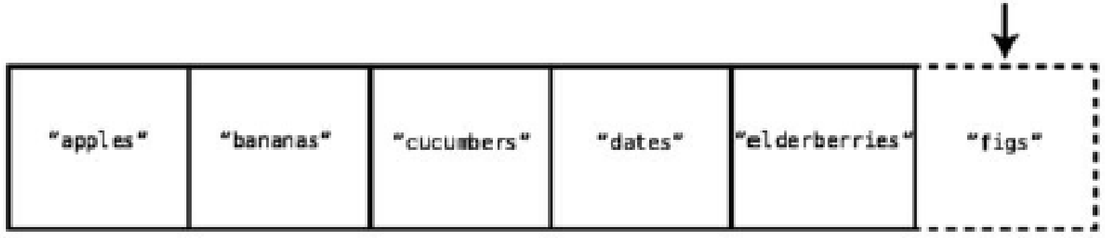


Step #5: Search index 4:



Only now that we've searched the entire set do we know that it's safe to insert "figs". And that brings us to our final step.

Step #6: Insert "figs" at the end of the set:



Insertion into a set in a **best-case scenario** will take N + 1 steps for N elements. This is because there are N steps of search to ensure that the value doesn't already exist within the set, and then one step for the actual insertion.

In a **worst-case scenario**, where we're inserting a value at the **beginning** of a set, the computer needs to search N cells to ensure that the set doesn't already contain that value, and then another N steps to shift all the data to the right, and another final step to insert the new value. That's a total of 2N + 1 steps.

SUMMARY

- Analyzing the number of steps an operation takes is key to understanding the **performance of data structures**. Choosing the right data structure can significantly affect a **program's efficiency.**
- This chapter introduces the importance of **time complexity analysis** of array and a set, depending on the application.
- The same analysis can be applied to compare different algorithms to ensure optimal speed and performance.